
MODULE *Directory*

```

*****
Directory.tla — formal specification AND proofs of the User-intent decision procedure for
MarkdownToLaTeX 1.0.0.

Subject. The User runs MarkdownToLaTeX 1.0.0 from a working directory DIR. Before any
conversion can begin, MarkdownToLaTeX must decide whether DIR is “ready” —
i.e. whether DIR contains a well-formed preferences.json at the canonical path
$DIR/preferences/preferences.json
with the required schema keys at the top level and a legal value for “document class”.

Verdict. The decision procedure produces exactly one of two terminal verdicts:
MUST_DO — DIR is ready; conversion may proceed. NOT_MUST_DO —
DIR is not ready; conversion must not start.

Semantics. Proof-state, not Kripke. The variable directory does not
describe the filesystem; it describes the * evidence we have
accumulated about the filesystem *. Every flag starts FALSE
(“nothing proven yet”) and may flip to TRUE only as the
corresponding check succeeds. A flag never flips back.

Termination. The procedure executes a fixed chain of nine checks. After
the last one | or after any failure | the procedure halts.

Disambiguation. To distinguish “not yet attempted” from “attempted and
refuted”, we keep a per – step attempted record alongside the
proof flags(option(a)per design discussion).

Author. Jean – Gabriel Cordier, with Claude.
Origin. Earlier draft Tue Aug 08 2023 by gcordier(single – step model).
This file. Kaizen step 1.3.0 | full chain, proof – state semantics,
MUST_DO/NOT_MUST_DO verdict, XOR – completeness theorem,
weak fairness, and TLAPS proof bodies for theorems T1–T6.
*****

```

EXTENDS *TLAPS*, *FiniteSets*, *Naturals*

CONSTANTS

LegalDocumentClass | the finite set of legal triples for the JSON value
at the “document class” key. Seeded with the single
default triple drawn from *scopeStatement_1_3_0.md*.
Extending this set later does not require any change
to this module.

CONSTANTS *LegalDocumentClass*

ASSUME *LegalDocumentClassIsFinite* $\triangleq$
 $\wedge$ *IsFiniteSet*(*LegalDocumentClass*)
 $\wedge$ $\forall t \in \text{LegalDocumentClass} :$
 $t \in [\text{class} : \text{STRING}, \text{size} : \text{STRING}, \text{paper} : \text{STRING}]$

Verdict alphabet.

$Verdict \triangleq \{ "UNKNOWN", "MUST_DO", "NOT_MUST_DO" \}$

Step alphabet. Nine ordered steps, named by what they prove.

$Step \triangleq \{$	<i>Step 1 : DIR itself</i>
"preferences_is_dir",	<i>Step 2 : DIR/preferences</i>
"preferences_has_file",	<i>Step 3 : preferences.json regular</i>
"json_author",	<i>Step 4a</i>
"json_email",	<i>Step 4b</i>
"json_name",	<i>Step 4c</i>
"json_main_font",	<i>Step 4d</i>
"json_document_class",	<i>Step 4e key presence</i>
"json_document_class_legal"	<i>Step 4f value in LegalDocumentClass</i>
$\}$	

Step ordering. The chain is linear; later steps may not be attempted until every earlier step has been proven.

$Predecessors(s) \triangleq$	
CASE $s = "is_dir"$	$\rightarrow \{ \}$
$\square \quad s = "preferences_is_dir"$	$\rightarrow \{ "is_dir" \}$
$\square \quad s = "preferences_has_file"$	$\rightarrow \{ "is_dir", "preferences_is_dir" \}$
$\square \quad s = "json_author"$	$\rightarrow \{ "is_dir", "preferences_is_dir", "preferences_has_file" \}$
$\square \quad s = "json_email"$	$\rightarrow \{ "is_dir", "preferences_is_dir", "preferences_has_file", "json_author" \}$
$\square \quad s = "json_name"$	$\rightarrow \{ "is_dir", "preferences_is_dir", "preferences_has_file", "json_author", "json_email" \}$
$\square \quad s = "json_main_font"$	$\rightarrow \{ "is_dir", "preferences_is_dir", "preferences_has_file", "json_author", "json_email", "json_name" \}$
$\square \quad s = "json_document_class"$	$\rightarrow \{ "is_dir", "preferences_is_dir", "preferences_has_file", "json_author", "json_email", "json_name", "json_main_font" \}$

$$\square \quad s = \text{"json_document_class_legal"} \rightarrow \{ \text{"is_dir"}, \text{"preferences_is_dir"}, \\ \text{"preferences_has_file"}, \\ \text{"json_author"}, \text{"json_email"}, \\ \text{"json_name"}, \text{"json_main_font"}, \\ \text{"json_document_class"} \}$$

VARIABLES

directory | *proof flags accumulated about DIR. Each leaf is FALSE initially and may flip to TRUE only via a SUCCESS action. Flags never flip back.*

attempted | *per – step boolean : TRUE iff that step s SUCCESS or FAILURE action has fired. Disambiguates “not yet” from “tried and refuted”.*

verdict — *UNKNOWN until the procedure halts; then exactly one of MUST_DO or NOT_MUST_DO.*

VARIABLES *directory, attempted, verdict*

$\text{vars} \triangleq \langle \text{directory}, \text{attempted}, \text{verdict} \rangle$

Type — the shape of the evidence record. Top-level fields mirror the four layers of the design suggestion (*DIR*, preferences/, the *JSON* file, the *JSON* contents); leaf fields are the boolean proof flags.

$$\begin{aligned} \text{Type} &\triangleq [\\ &\quad \text{is_dir} : \text{BOOLEAN} , && \text{Step 1} \\ &\quad \text{preferences} : [\\ &\quad \quad \text{is_dir} : \text{BOOLEAN} , && \text{Step 2} \\ &\quad \quad \text{has_file} : \text{BOOLEAN} && \text{Step 3} \\ &\quad], \\ &\quad \text{preferences_json} : [\\ &\quad \quad \text{keys} : [&& \text{Step 4a .. 4e} \\ &\quad \quad \quad \text{author} : \text{BOOLEAN} , \\ &\quad \quad \quad \text{email} : \text{BOOLEAN} , \\ &\quad \quad \quad \text{name} : \text{BOOLEAN} , \\ &\quad \quad \quad \text{main_font} : \text{BOOLEAN} , \\ &\quad \quad \quad \text{document_class} : \text{BOOLEAN} \\ &\quad], \\ &\quad \text{document_class_legal} : \text{BOOLEAN} && \text{Step 4f} \\ &] \\ &] \\ \text{TypeOK} &\triangleq \end{aligned}$$

$$\begin{aligned}
&\wedge \text{ directory} \in \text{Type} \\
&\wedge \text{ attempted} \in [\text{Step} \rightarrow \text{BOOLEAN}] \\
&\wedge \text{ verdict} \in \text{Verdict}
\end{aligned}$$

Convenience: the proof-flag value for a given step.

$$\begin{aligned}
\text{Proven}(s) &\triangleq \\
&\text{CASE } s = \text{"is_dir"} && \rightarrow \text{directory.is_dir} \\
&\quad \square \text{ } s = \text{"preferences.is_dir"} && \rightarrow \text{directory.preferences.is_dir} \\
&\quad \square \text{ } s = \text{"preferences.has_file"} && \rightarrow \text{directory.preferences.has_file} \\
&\quad \square \text{ } s = \text{"json_author"} && \rightarrow \text{directory.preferences_json.keys.author} \\
&\quad \square \text{ } s = \text{"json_email"} && \rightarrow \text{directory.preferences_json.keys.email} \\
&\quad \square \text{ } s = \text{"json_name"} && \rightarrow \text{directory.preferences_json.keys.name} \\
&\quad \square \text{ } s = \text{"json_main_font"} && \rightarrow \text{directory.preferences_json.keys.main_font} \\
&\quad \square \text{ } s = \text{"json_document_class"} && \rightarrow \text{directory.preferences_json.keys.document_class} \\
&\quad \square \text{ } s = \text{"json_document_class_legal"} && \rightarrow \text{directory.preferences_json.document_class_legal}
\end{aligned}$$

AllProven — every flag TRUE. Equivalent to *MUST_DO* at termination.

$$\text{AllProven} \triangleq \forall s \in \text{Step} : \text{Proven}(s)$$

INIT

Nothing has been proven, nothing has been attempted, the verdict is open.

$$\begin{aligned}
\text{InitDirectory} &\triangleq [\\
&\text{is_dir} \mapsto \text{FALSE}, \\
&\text{preferences} \mapsto [\text{is_dir} \mapsto \text{FALSE}, \text{has_file} \mapsto \text{FALSE}], \\
&\text{preferences_json} \mapsto [\\
&\quad \text{keys} \mapsto [\text{author} \mapsto \text{FALSE}, \\
&\quad \text{email} \mapsto \text{FALSE}, \\
&\quad \text{name} \mapsto \text{FALSE}, \\
&\quad \text{main_font} \mapsto \text{FALSE}, \\
&\quad \text{document_class} \mapsto \text{FALSE}], \\
&\quad \text{document_class_legal} \mapsto \text{FALSE} \\
&] \\
&]
\end{aligned}$$

$$\begin{aligned}
\text{Init} &\triangleq \\
&\wedge \text{ directory} = \text{InitDirectory} \\
&\wedge \text{ attempted} = [s \in \text{Step} \mapsto \text{FALSE}] \\
&\wedge \text{ verdict} = \text{"UNKNOWN"}
\end{aligned}$$

GUARDS

A step is *Ready* when every predecessor has been proven, the step itself has not been attempted, and the verdict is still *UNKNOWN*.

$$\begin{aligned}
 \text{Ready}(s) &\triangleq \\
 &\wedge \text{verdict} = \text{"UNKNOWN"} \\
 &\wedge \neg \text{attempted}[s] \\
 &\wedge \forall p \in \text{Predecessors}(s) : \text{Proven}(p)
 \end{aligned}$$

SUCCESS ACTIONS

Each success action :

- (1) requires *Ready*(s);
 - (2) flips *Proven*(s) from FALSE to TRUE in directory ;
 - (3) marks *attempted*[s] := TRUE;
 - (4) leaves verdict *UNKNOWN* unless this is the final step
 (*json_document_class_legal*), in which case verdict := *MUST_DO*
- =====

Helpers for record updates

$$\begin{aligned}
 \text{WithProvenIsDir} &\triangleq \\
 &[\text{directory} \text{ EXCEPT } !.\text{is_dir} = \text{TRUE}] \\
 \\
 \text{WithProvenPreferencesIsDir} &\triangleq \\
 &[\text{directory} \text{ EXCEPT } !.\text{preferences.is_dir} = \text{TRUE}] \\
 \\
 \text{WithProvenPreferencesHasFile} &\triangleq \\
 &[\text{directory} \text{ EXCEPT } !.\text{preferences.has_file} = \text{TRUE}] \\
 \\
 \text{WithProvenJsonKey}(k) &\triangleq \\
 &[\text{directory} \text{ EXCEPT } !.\text{preferences_json.keys}[k] = \text{TRUE}] \\
 \\
 \text{WithProvenDocClassLegal} &\triangleq \\
 &[\text{directory} \text{ EXCEPT } !.\text{preferences_json.document_class_legal} = \text{TRUE}]
 \end{aligned}$$

Success actions

$$\begin{aligned}
 \text{Check_IS_DIR_SUCCESS} &\triangleq \\
 &\wedge \text{Ready}(\text{"is_dir"}) \\
 &\wedge \text{directory}' = \text{WithProvenIsDir} \\
 &\wedge \text{attempted}' = [\text{attempted} \text{ EXCEPT } ![\text{"is_dir"}] = \text{TRUE}] \\
 &\wedge \text{UNCHANGED } \text{verdict} \\
 \\
 \text{Check_PREFS_IS_DIR_SUCCESS} &\triangleq \\
 &\wedge \text{Ready}(\text{"preferences_is_dir"}) \\
 &\wedge \text{directory}' = \text{WithProvenPreferencesIsDir}
 \end{aligned}$$

$\wedge attempted' = [attempted \text{ EXCEPT } !["preferences_is_dir"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_PREFS_HAS_FILE_SUCCESS \triangleq$
 $\wedge Ready("preferences_has_file")$
 $\wedge directory' = WithProvenPreferencesHasFile$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["preferences_has_file"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_JSON_AUTHOR_SUCCESS \triangleq$
 $\wedge Ready("json_author")$
 $\wedge directory' = WithProvenJsonKey("author")$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_author"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_JSON_EMAIL_SUCCESS \triangleq$
 $\wedge Ready("json_email")$
 $\wedge directory' = WithProvenJsonKey("email")$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_email"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_JSON_NAME_SUCCESS \triangleq$
 $\wedge Ready("json_name")$
 $\wedge directory' = WithProvenJsonKey("name")$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_name"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_JSON_MAIN_FONT_SUCCESS \triangleq$
 $\wedge Ready("json_main_font")$
 $\wedge directory' = WithProvenJsonKey("main_font")$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_main_font"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

$Check_JSON_DOC_CLASS_SUCCESS \triangleq$
 $\wedge Ready("json_document_class")$
 $\wedge directory' = WithProvenJsonKey("document_class")$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_document_class"] = \text{TRUE}]$
 $\wedge \text{UNCHANGED } verdict$

Final success | flips the verdict to MUST_DO.

$Check_JSON_DOC_CLASS_LEGAL_SUCCESS \triangleq$
 $\wedge Ready("json_document_class_legal")$
 $\wedge directory' = WithProvenDocClassLegal$
 $\wedge attempted' = [attempted \text{ EXCEPT } !["json_document_class_legal"] = \text{TRUE}]$
 $\wedge verdict' = \text{"MUST_DO"}$

FAILURE ACTIONS

Any step that is *Ready* may instead fail. A failure :

- (1) does not flip the proof flag;
- (2) marks $attempted[s] := \text{TRUE}$;
- (3) sets $verdict := \text{NOT_MUST_DO}$, terminating the procedure.

We define a single parameterised failure action and instantiate it for each step in the disjunction below.

$$\begin{aligned}
 Fail(s) &\triangleq \\
 &\wedge Ready(s) \\
 &\wedge \text{UNCHANGED } directory \\
 &\wedge attempted' = [attempted \text{ EXCEPT } ![s] = \text{TRUE}] \\
 &\wedge verdict' = \text{"NOT_MUST_DO"}
 \end{aligned}$$

NEXT

$$\begin{aligned}
 Next &\triangleq \\
 &\vee Check_IS_DIR_SUCCESS \\
 &\vee Check_PREFS_IS_DIR_SUCCESS \\
 &\vee Check_PREFS_HAS_FILE_SUCCESS \\
 &\vee Check_JSON_AUTHOR_SUCCESS \\
 &\vee Check_JSON_EMAIL_SUCCESS \\
 &\vee Check_JSON_NAME_SUCCESS \\
 &\vee Check_JSON_MAIN_FONT_SUCCESS \\
 &\vee Check_JSON_DOC_CLASS_SUCCESS \\
 &\vee Check_JSON_DOC_CLASS_LEGAL_SUCCESS \\
 &\vee \exists s \in Step : Fail(s)
 \end{aligned}$$

FAIRNESS/SPEC

Weak fairness on *Next* ensures the procedure cannot stall forever in an intermediate state : if a step is continuously *Ready*, it must eventually fire (either as success or failure).

$$Spec \triangleq Init \wedge \Box [Next]_{vars} \wedge WF_{vars}(Next)$$

INVARIANTS

$I1 \mid type - correctness.$

$$Inv_TypeOK \triangleq TypeOK$$

I2 | monotonicity of proof flags is enforced action – by – action; we state it here as a stuttering invariant on consecutive states.

$$\begin{aligned} \text{Inv_FlagsMonotone} &\triangleq \\ \square[\forall s \in \text{Step} : \text{Proven}(s) \Rightarrow \text{Proven}(s')]_{\text{vars}} \end{aligned}$$

I3 | attempted is monotone too : once attempted, always attempted.

$$\begin{aligned} \text{Inv_AttemptedMonotone} &\triangleq \\ \square[\forall s \in \text{Step} : \text{attempted}[s] \Rightarrow \text{attempted}'[s]]_{\text{vars}} \end{aligned}$$

I4 | the verdict, once decided, is permanent.

$$\begin{aligned} \text{Inv_VerdictStable} &\triangleq \\ \square[\text{verdict} \neq \text{"UNKNOWN"} \Rightarrow \text{verdict}' = \text{verdict}]_{\text{vars}} \end{aligned}$$

I5 | soundness of MUST_DO : it can only be set when every flag is TRUE.

$$\begin{aligned} \text{Inv_MustDoSound} &\triangleq \\ (\text{verdict} = \text{"MUST_DO"}) \Rightarrow \text{AllProven} \end{aligned}$$

I6 | soundness of NOT_MUST_DO : it implies at least one flag is FALSE.

$$\begin{aligned} \text{Inv_NotMustDoSound} &\triangleq \\ (\text{verdict} = \text{"NOT_MUST_DO"}) \Rightarrow \neg \text{AllProven} \end{aligned}$$

H | auxiliary state invariant bridging proof flags and attempted The success actions are the only paths to $\text{Proven}(s) = \text{TRUE}$, and each such action also sets $\text{attempted}[s] := \text{TRUE}$; therefore $\text{Proven}(s)$ implies $\text{attempted}[s]$ in every reachable state. Used to establish T6 (NotMustDoSoundness).

$$\begin{aligned} \text{Inv_ProvenAttempted} &\triangleq \\ \forall s \in \text{Step} : \text{Proven}(s) \Rightarrow \text{attempted}[s] \end{aligned}$$

DEFINITIONS USED IN THEOREMS

These definitions are stated up front so that every theorem below can refer to them without forward references.

Termination predicate. The procedure has halted when verdict has been pinned to one of the two terminal values.

$$\text{Terminated} \triangleq \text{verdict} \in \{\text{"MUST_DO"}, \text{"NOT_MUST_DO"}\}$$

XOR – completeness predicate (T4). At termination, exactly one of MUST_DO and NOT_MUST_DO holds; spelled out as a propositional XOR over the two string equalities.

$$\begin{aligned} \text{XORComplete} &\triangleq \\ \text{Terminated} \Rightarrow \\ & ((\text{verdict} = \text{"MUST_DO"} \quad \wedge \text{verdict} \neq \text{"NOT_MUST_DO"}) \\ & \vee (\text{verdict} = \text{"NOT_MUST_DO"} \wedge \text{verdict} \neq \text{"MUST_DO"})) \end{aligned}$$

Termination measure : how many steps have been attempted so far. The chain has 9 steps; each non – stutter Next – action increments this by exactly one. Used in the leads – to chain that proves T3.

$$AttemptedCount \triangleq Cardinality(\{s \in Step : attempted[s]\})$$

Family of intermediate predicates for the leads – to chain(T3). The procedure terminates, OR at least k steps have been attempted.

$$P(k) \triangleq Terminated \vee AttemptedCount \geq k$$

THEOREMS

T1, T2, T4, T5, T6 are safety properties; T3 is the lone liveness property and depends on theWF_vars(Next)fairness clause.

Order below : T1, T2, helper, T5, T6, T4, supporting machinery for T3, T3. This order mirrors the proof – dependency DAG (see properties.figures.1.3.0.tex, Figure 2) and ensures every reference is backward.

T1 | Type – correctness is preserved.

Standard inductive – invariant proof. We dispatch on the structure of Next, then on each disjunct.

THEOREM $TypeCorrectness \triangleq Spec \Rightarrow \Box TypeOK$

$\langle 1 \rangle 1. Init \Rightarrow TypeOK$

BY DEF *Init, InitDirectory, TypeOK, Type, Verdict, Step*

$\langle 1 \rangle 2. TypeOK \wedge [Next]_{vars} \Rightarrow TypeOK'$

$\langle 2 \rangle$ SUFFICES ASSUME $TypeOK, [Next]_{vars}$
PROVE $TypeOK'$

OBVIOUS

$\langle 2 \rangle$ USE DEF *TypeOK, Type, Verdict, Step*

$\langle 2 \rangle 1.$ CASE UNCHANGED *vars*

BY $\langle 2 \rangle 1$ DEF *vars*

$\langle 2 \rangle 2.$ CASE *Check_IS_DIR_SUCCESS*

BY $\langle 2 \rangle 2$ DEF *Check_IS_DIR_SUCCESS, WithProvenIsDir*

$\langle 2 \rangle 3.$ CASE *Check_PREFS_IS_DIR_SUCCESS*

BY $\langle 2 \rangle 3$ DEF *Check_PREFS_IS_DIR_SUCCESS, WithProvenPreferencesIsDir*

$\langle 2 \rangle 4.$ CASE *Check_PREFS_HAS_FILE_SUCCESS*

BY $\langle 2 \rangle 4$ DEF *Check_PREFS_HAS_FILE_SUCCESS, WithProvenPreferencesHasFile*

$\langle 2 \rangle 5.$ CASE *Check_JSON_AUTHOR_SUCCESS*

BY $\langle 2 \rangle 5$ DEF *Check_JSON_AUTHOR_SUCCESS, WithProvenJsonKey*

$\langle 2 \rangle 6.$ CASE *Check_JSON_EMAIL_SUCCESS*

BY $\langle 2 \rangle 6$ DEF *Check_JSON_EMAIL_SUCCESS, WithProvenJsonKey*

$\langle 2 \rangle 7.$ CASE *Check_JSON_NAME_SUCCESS*
 BY $\langle 2 \rangle 7$ DEF *Check_JSON_NAME_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 8.$ CASE *Check_JSON_MAIN_FONT_SUCCESS*
 BY $\langle 2 \rangle 8$ DEF *Check_JSON_MAIN_FONT_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 9.$ CASE *Check_JSON_DOC_CLASS_SUCCESS*
 BY $\langle 2 \rangle 9$ DEF *Check_JSON_DOC_CLASS_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 10.$ CASE *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*
 BY $\langle 2 \rangle 10$ DEF *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*, *WithProvenDocClassLegal*
 $\langle 2 \rangle 11.$ CASE $\exists s \in \text{Step} : \text{Fail}(s)$
 BY $\langle 2 \rangle 11$ DEF *Fail*
 $\langle 2 \rangle$ QED
 BY $\langle 2 \rangle 1, \langle 2 \rangle 2, \langle 2 \rangle 3, \langle 2 \rangle 4, \langle 2 \rangle 5, \langle 2 \rangle 6, \langle 2 \rangle 7, \langle 2 \rangle 8, \langle 2 \rangle 9, \langle 2 \rangle 10, \langle 2 \rangle 11$
 DEF *Next*
 $\langle 1 \rangle$ QED
 BY $\langle 1 \rangle 1, \langle 1 \rangle 2$, PTL DEF *Spec*

T2 | *Verdict stability : once decided(UNKNOWN), the verdict cannot change. Every action that * changes * the verdict is guarded by Ready(-), which itself requires verdict = "UNKNOWN". So if the pre - state has a decided verdict, the only enabled transition is a stutter, which preserves it.*

THEOREM *VerdictStability* $\triangleq \text{Spec} \Rightarrow \text{Inv_VerdictStable}$
 $\langle 1 \rangle 1.$ $[\text{Next}]_{\text{vars}} \wedge \text{verdict} \neq \text{"UNKNOWN"} \Rightarrow \text{verdict}' = \text{verdict}$
 $\langle 2 \rangle$ SUFFICES ASSUME $[\text{Next}]_{\text{vars}}, \text{verdict} \neq \text{"UNKNOWN"}$
 PROVE $\text{verdict}' = \text{verdict}$
 OBVIOUS
 $\langle 2 \rangle 1.$ CASE UNCHANGED *vars*
 BY $\langle 2 \rangle 1$ DEF *vars*
 $\langle 2 \rangle 2.$ CASE *Next*
 $\langle 3 \rangle$ USE DEF *Ready*
 $\langle 3 \rangle 1.$ CASE *Check_IS_DIR_SUCCESS*
 BY $\langle 3 \rangle 1, \langle 2 \rangle 2$ DEF *Check_IS_DIR_SUCCESS*
 $\langle 3 \rangle 2.$ CASE *Check_PREFS_IS_DIR_SUCCESS*
 BY $\langle 3 \rangle 2, \langle 2 \rangle 2$ DEF *Check_PREFS_IS_DIR_SUCCESS*
 $\langle 3 \rangle 3.$ CASE *Check_PREFS_HAS_FILE_SUCCESS*
 BY $\langle 3 \rangle 3, \langle 2 \rangle 2$ DEF *Check_PREFS_HAS_FILE_SUCCESS*
 $\langle 3 \rangle 4.$ CASE *Check_JSON_AUTHOR_SUCCESS*
 BY $\langle 3 \rangle 4, \langle 2 \rangle 2$ DEF *Check_JSON_AUTHOR_SUCCESS*
 $\langle 3 \rangle 5.$ CASE *Check_JSON_EMAIL_SUCCESS*
 BY $\langle 3 \rangle 5, \langle 2 \rangle 2$ DEF *Check_JSON_EMAIL_SUCCESS*
 $\langle 3 \rangle 6.$ CASE *Check_JSON_NAME_SUCCESS*
 BY $\langle 3 \rangle 6, \langle 2 \rangle 2$ DEF *Check_JSON_NAME_SUCCESS*
 $\langle 3 \rangle 7.$ CASE *Check_JSON_MAIN_FONT_SUCCESS*

```

    BY ⟨3⟩7, ⟨2⟩2 DEF Check_JSON_MAIN_FONT_SUCCESS
  ⟨3⟩8.CASE Check_JSON_DOC_CLASS_SUCCESS
    BY ⟨3⟩8, ⟨2⟩2 DEF Check_JSON_DOC_CLASS_SUCCESS
  ⟨3⟩9.CASE Check_JSON_DOC_CLASS_LEGAL_SUCCESS
    Action requires Ready, which requires verdict = "UNKNOWN".
    Contradiction with the assumption verdict ≠ "UNKNOWN".
    BY ⟨3⟩9, ⟨2⟩2 DEF Check_JSON_DOC_CLASS_LEGAL_SUCCESS
  ⟨3⟩10.CASE ∃ s ∈ Step : Fail(s)
    BY ⟨3⟩10, ⟨2⟩2 DEF Fail
  ⟨3⟩ QED
  BY ⟨2⟩2, ⟨3⟩1, ⟨3⟩2, ⟨3⟩3, ⟨3⟩4, ⟨3⟩5, ⟨3⟩6, ⟨3⟩7, ⟨3⟩8, ⟨3⟩9, ⟨3⟩10
    DEF Next
  ⟨2⟩ QED BY ⟨2⟩1, ⟨2⟩2 DEF vars
  ⟨1⟩ QED
  BY ⟨1⟩1, PTL DEF Spec, Inv_VerdictStable

```

Helper lemma : Inv_ProvenAttempted is preserved.

Each success action is the only way to flip a proof flag, and it always also flips the corresponding attempted entry. Failure actions and stutter never flip a proof flag. So Proven(s) attempted[s] is inductive.

We need this lemma for T6(NotMustDoSoundness) : it lets us deduce that the step which fired Fail(−) had its flag still FALSE at the moment of failure | i.e. the step s Proven(−) was never set, so AllProven was refuted.

```

LEMMA ProvenAttemptedInvariant  $\triangleq$  Spec  $\Rightarrow$   $\Box$ Inv_ProvenAttempted
  ⟨1⟩1. Init  $\Rightarrow$  Inv_ProvenAttempted
    BY DEF Init, InitDirectory, Inv_ProvenAttempted, Proven, Step
  ⟨1⟩2. TypeOK  $\wedge$  Inv_ProvenAttempted  $\wedge$  [Next]vars  $\Rightarrow$  Inv_ProvenAttempted'
    ⟨2⟩ SUFFICES ASSUME TypeOK, Inv_ProvenAttempted, [Next]vars,
      NEW s ∈ Step, Proven(s)'
      PROVE attempted'[s]
    BY DEF Inv_ProvenAttempted
  ⟨2⟩ USE DEF TypeOK, Type, Step, Proven, Inv_ProvenAttempted
  ⟨2⟩1.CASE UNCHANGED vars
    BY ⟨2⟩1 DEF vars
  ⟨2⟩2.CASE Check_IS_DIR_SUCCESS
    Action sets directory'.is_dir = TRUE and attempted'["is_dir"] = TRUE,
    and leaves all other proof flags and other attempted entries unchanged.
    BY ⟨2⟩2 DEF Check_IS_DIR_SUCCESS, WithProvenIsDir
  ⟨2⟩3.CASE Check_PREFS_IS_DIR_SUCCESS
    BY ⟨2⟩3 DEF Check_PREFS_IS_DIR_SUCCESS, WithProvenPreferencesIsDir
  ⟨2⟩4.CASE Check_PREFS_HAS_FILE_SUCCESS

```

BY $\langle 2 \rangle 4$ DEF *Check_PREFS_HAS_FILE_SUCCESS*, *WithProvenPreferencesHasFile*
 $\langle 2 \rangle 5$.CASE *Check_JSON_AUTHOR_SUCCESS*
 BY $\langle 2 \rangle 5$ DEF *Check_JSON_AUTHOR_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 6$.CASE *Check_JSON_EMAIL_SUCCESS*
 BY $\langle 2 \rangle 6$ DEF *Check_JSON_EMAIL_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 7$.CASE *Check_JSON_NAME_SUCCESS*
 BY $\langle 2 \rangle 7$ DEF *Check_JSON_NAME_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 8$.CASE *Check_JSON_MAIN_FONT_SUCCESS*
 BY $\langle 2 \rangle 8$ DEF *Check_JSON_MAIN_FONT_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 9$.CASE *Check_JSON_DOC_CLASS_SUCCESS*
 BY $\langle 2 \rangle 9$ DEF *Check_JSON_DOC_CLASS_SUCCESS*, *WithProvenJsonKey*
 $\langle 2 \rangle 10$.CASE *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*
 BY $\langle 2 \rangle 10$ DEF *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*, *WithProvenDocClassLegal*
 $\langle 2 \rangle 11$.CASE $\exists t \in \text{Step} : \text{Fail}(t)$
 Failure actions leave *directory* unchanged. So $\text{Proven}(s) = \text{Proven}(s)$.
 And attempted only grows: $\text{attempted}'[s] = \text{TRUE}$ if it was TRUE.
 BY $\langle 2 \rangle 11$ DEF *Fail*
 $\langle 2 \rangle$ QED
 BY $\langle 2 \rangle 1, \langle 2 \rangle 2, \langle 2 \rangle 3, \langle 2 \rangle 4, \langle 2 \rangle 5, \langle 2 \rangle 6, \langle 2 \rangle 7, \langle 2 \rangle 8, \langle 2 \rangle 9, \langle 2 \rangle 10, \langle 2 \rangle 11$
 DEF *Next*
 $\langle 1 \rangle$ QED
 BY $\langle 1 \rangle 1, \langle 1 \rangle 2$, *TypeCorrectness*, *PTL* DEF *Spec*

T5 — Soundness of *MUST_DO*: $\text{verdict} = \text{MUST_DO} \implies \text{AllProven}$.

Inductive. *Init* has $\text{verdict} = \text{UNKNOWN}$, vacuous.

For the step case, suppose $\text{verdict}' = \text{MUST_DO}$. Either: (a) $\text{verdict} = \text{MUST_DO}$ and the action is a stutter or one whose guard

Ready($_$) requires $\text{verdict} = \text{UNKNOWN}$ — the latter is therefore impossible. Stutter preserves *AllProven* by *IH* and *UNCHANGED*.

(b) $\text{verdict} = \text{UNKNOWN}$ in the pre-state and the action is one that sets $\text{verdict} := \text{MUST_DO}$. The only such action is *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*, whose guard requires every predecessor proven (*i.e.* flags 1 .. 8) and which itself proves flag 9. So *AllProven'* holds.

No other action can produce $\text{verdict}' = \text{MUST_DO}$.

THEOREM *MustDoSoundness* $\triangleq \text{Spec} \Rightarrow \Box \text{Inv_MustDoSound}$

$\langle 1 \rangle 1$. *Init* $\Rightarrow \text{Inv_MustDoSound}$

BY DEF *Init*, *Inv_MustDoSound*, *Verdict*

$\langle 1 \rangle 2$. *TypeOK* $\wedge \text{Inv_MustDoSound} \wedge [\text{Next}]_{\text{vars}} \Rightarrow \text{Inv_MustDoSound}'$

$\langle 2 \rangle$ SUFFICES ASSUME *TypeOK*, *Inv_MustDoSound*, $[\text{Next}]_{\text{vars}}$,
 $\text{verdict}' = \text{"MUST_DO"}$

PROVE *AllProven'*

BY DEF *Inv_MustDoSound*

$\langle 2 \rangle$ USE DEF *TypeOK*, *Type*, *Step*, *Verdict*, *Ready*, *AllProven*, *Proven*,
Inv_MustDoSound
 $\langle 2 \rangle$ 1.CASE UNCHANGED *vars*
BY $\langle 2 \rangle$ 1 DEF *vars*
 $\langle 2 \rangle$ 2.CASE *Check_IS_DIR_SUCCESS*
This action has UNCHANGED *verdict*. So *verdict* = "MUST_DO" pre.
But *Ready*("is_dir") demands *verdict* = "UNKNOWN". Contradiction.
BY $\langle 2 \rangle$ 2 DEF *Check_IS_DIR_SUCCESS*
 $\langle 2 \rangle$ 3.CASE *Check_PREFS_IS_DIR_SUCCESS*
BY $\langle 2 \rangle$ 3 DEF *Check_PREFS_IS_DIR_SUCCESS*
 $\langle 2 \rangle$ 4.CASE *Check_PREFS_HAS_FILE_SUCCESS*
BY $\langle 2 \rangle$ 4 DEF *Check_PREFS_HAS_FILE_SUCCESS*
 $\langle 2 \rangle$ 5.CASE *Check_JSON_AUTHOR_SUCCESS*
BY $\langle 2 \rangle$ 5 DEF *Check_JSON_AUTHOR_SUCCESS*
 $\langle 2 \rangle$ 6.CASE *Check_JSON_EMAIL_SUCCESS*
BY $\langle 2 \rangle$ 6 DEF *Check_JSON_EMAIL_SUCCESS*
 $\langle 2 \rangle$ 7.CASE *Check_JSON_NAME_SUCCESS*
BY $\langle 2 \rangle$ 7 DEF *Check_JSON_NAME_SUCCESS*
 $\langle 2 \rangle$ 8.CASE *Check_JSON_MAIN_FONT_SUCCESS*
BY $\langle 2 \rangle$ 8 DEF *Check_JSON_MAIN_FONT_SUCCESS*
 $\langle 2 \rangle$ 9.CASE *Check_JSON_DOC_CLASS_SUCCESS*
BY $\langle 2 \rangle$ 9 DEF *Check_JSON_DOC_CLASS_SUCCESS*
 $\langle 2 \rangle$ 10.CASE *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*
Ready("json_document_class_legal") requires every predecessor
proven. The action itself proves the last flag. The other eight
flags survive the EXCEPT update unchanged.
BY $\langle 2 \rangle$ 10 DEF *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*,
WithProvenDocClassLegal, *Predecessors*
 $\langle 2 \rangle$ 11.CASE $\exists s \in \text{Step} : \text{Fail}(s)$
Fail sets *verdict*' = "NOT_MUST_DO", contradicting *verdict*' = "MUST_DO".
BY $\langle 2 \rangle$ 11 DEF *Fail*
 $\langle 2 \rangle$ QED
BY $\langle 2 \rangle$ 1, $\langle 2 \rangle$ 2, $\langle 2 \rangle$ 3, $\langle 2 \rangle$ 4, $\langle 2 \rangle$ 5, $\langle 2 \rangle$ 6, $\langle 2 \rangle$ 7, $\langle 2 \rangle$ 8, $\langle 2 \rangle$ 9, $\langle 2 \rangle$ 10, $\langle 2 \rangle$ 11
DEF *Next*
 $\langle 1 \rangle$ QED
BY $\langle 1 \rangle$ 1, $\langle 1 \rangle$ 2, *TypeCorrectness*, *PTL* DEF *Spec*

T6 — Soundness of *NOT_MUST_DO*: *verdict* = *NOT_MUST_DO* $\neg$ *AllProven*.

The only action that sets *verdict* := *NOT_MUST_DO* is *Fail*(*s*) for some *s*. *Fail*(*s*) is guarded by *Ready*(*s*), which requires $\neg \text{attempted}[s]$. Combined with the helper invariant *Inv_ProvenAttempted* (*Proven*(*s*) *attempted*[*s*]), we deduce $\neg \text{Proven}(s)$ at the moment of failure. Fail leaves *directory* unchanged, so *Proven*(*s*) as well — i.e. $\neg \text{AllProven}'$.

Verdict stability (T2) then carries this forward: once $verdict = NOT_MUST_DO$, no action fires that would re-prove flag s , so $\neg Proven(s)$ is permanent and $\neg AllProven$ holds at every subsequent state.

THEOREM $NotMustDoSoundness \triangleq Spec \Rightarrow \Box Inv_NotMustDoSound$

$\langle 1 \rangle 1. Init \Rightarrow Inv_NotMustDoSound$
 BY DEF $Init, Inv_NotMustDoSound, Verdict$

$\langle 1 \rangle 2. TypeOK \wedge Inv_ProvenAttempted \wedge Inv_NotMustDoSound \wedge [Next]_{vars} \Rightarrow Inv_NotMustDoSound'$

$\langle 2 \rangle$ SUFFICES ASSUME $TypeOK, Inv_ProvenAttempted, Inv_NotMustDoSound, [Next]_{vars}, verdict' = "NOT_MUST_DO"$
 PROVE $\neg AllProven'$
 BY DEF $Inv_NotMustDoSound$

$\langle 2 \rangle$ USE DEF $TypeOK, Type, Step, Verdict, Ready, AllProven, Proven, Inv_ProvenAttempted, Inv_NotMustDoSound$

$\langle 2 \rangle 1.$ CASE UNCHANGED $vars$
 $verdict' = verdict = "NOT_MUST_DO"$ pre-state. By $IH \neg AllProven$, and $directory$ unchanged, so $AllProven$.

BY $\langle 2 \rangle 1$ DEF $vars$

$\langle 2 \rangle 2.$ CASE $Check_IS_DIR_SUCCESS$
 UNCHANGED $verdict$, so $verdict = "NOT_MUST_DO"$ pre. But $Ready$ demands $verdict = "UNKNOWN"$. Contradiction.

BY $\langle 2 \rangle 2$ DEF $Check_IS_DIR_SUCCESS$

$\langle 2 \rangle 3.$ CASE $Check_PREFS_IS_DIR_SUCCESS$
 BY $\langle 2 \rangle 3$ DEF $Check_PREFS_IS_DIR_SUCCESS$

$\langle 2 \rangle 4.$ CASE $Check_PREFS_HAS_FILE_SUCCESS$
 BY $\langle 2 \rangle 4$ DEF $Check_PREFS_HAS_FILE_SUCCESS$

$\langle 2 \rangle 5.$ CASE $Check_JSON_AUTHOR_SUCCESS$
 BY $\langle 2 \rangle 5$ DEF $Check_JSON_AUTHOR_SUCCESS$

$\langle 2 \rangle 6.$ CASE $Check_JSON_EMAIL_SUCCESS$
 BY $\langle 2 \rangle 6$ DEF $Check_JSON_EMAIL_SUCCESS$

$\langle 2 \rangle 7.$ CASE $Check_JSON_NAME_SUCCESS$
 BY $\langle 2 \rangle 7$ DEF $Check_JSON_NAME_SUCCESS$

$\langle 2 \rangle 8.$ CASE $Check_JSON_MAIN_FONT_SUCCESS$
 BY $\langle 2 \rangle 8$ DEF $Check_JSON_MAIN_FONT_SUCCESS$

$\langle 2 \rangle 9.$ CASE $Check_JSON_DOC_CLASS_SUCCESS$
 BY $\langle 2 \rangle 9$ DEF $Check_JSON_DOC_CLASS_SUCCESS$

$\langle 2 \rangle 10.$ CASE $Check_JSON_DOC_CLASS_LEGAL_SUCCESS$
 This action sets $verdict' = "MUST_DO"$, not $"NOT_MUST_DO"$.

BY $\langle 2 \rangle 10$ DEF $Check_JSON_DOC_CLASS_LEGAL_SUCCESS$

$\langle 2 \rangle 11.$ CASE $\exists s \in Step : Fail(s)$
 The witnessing s has $Ready(s)$, hence $\neg attempted[s]$. By the helper invariant $Inv_ProvenAttempted, \neg Proven(s)$. Fail leaves directory unchanged, so $\neg Proven(s)'$ — i.e. $\neg AllProven'$.

BY $\langle 2 \rangle 11$ DEF *Fail*
 $\langle 2 \rangle$ QED
 BY $\langle 2 \rangle 1, \langle 2 \rangle 2, \langle 2 \rangle 3, \langle 2 \rangle 4, \langle 2 \rangle 5, \langle 2 \rangle 6, \langle 2 \rangle 7, \langle 2 \rangle 8, \langle 2 \rangle 9, \langle 2 \rangle 10, \langle 2 \rangle 11$
 DEF *Next*
 $\langle 1 \rangle$ QED
 BY $\langle 1 \rangle 1, \langle 1 \rangle 2, \textit{TypeCorrectness}, \textit{ProvenAttemptedInvariant}, \textit{PTL}$ DEF *Spec*

T4 — XOR-completeness: at termination, exactly one of *MUST_DO* and *NOT_MUST_DO* holds. Trivially true given that “MUST_DO” and “NOT_MUST_DO” are distinct strings: the structure of *XORComplete* reduces to a tautology in the verdict alphabet.

THEOREM *XORCompleteness* $\triangleq \textit{Spec} \Rightarrow \Box(\textit{Terminated} \Rightarrow \textit{XORComplete})$
 $\langle 1 \rangle 1. \forall v \in \textit{Verdict} :$
 $(v \in \{ \text{“MUST_DO”}, \text{“NOT_MUST_DO”} \})$
 $\Rightarrow ((v = \text{“MUST_DO”} \quad \wedge \quad v \neq \text{“NOT_MUST_DO”})$
 $\quad \vee \quad (v = \text{“NOT_MUST_DO”} \quad \wedge \quad v \neq \text{“MUST_DO”}))$
 BY DEF *Verdict*
 $\langle 1 \rangle 2. \Box \textit{TypeOK} \Rightarrow \Box(\textit{Terminated} \Rightarrow \textit{XORComplete})$
 $\langle 2 \rangle$ SUFFICES ASSUME *TypeOK*, *Terminated* PROVE *XORComplete*
 BY *PTL*
 $\langle 2 \rangle$ QED
 BY $\langle 1 \rangle 1$ DEF *TypeOK*, *Terminated*, *XORComplete*
 $\langle 1 \rangle$ QED
 BY $\langle 1 \rangle 2, \textit{TypeCorrectness}, \textit{PTL}$ DEF *Spec*

T3 — *Termination* (LIVENESS).

Strategy. From any non-terminated state with k attempted steps ($0 \leq k < 9$), the next-in-chain step $s_{\{k+1\}}$ is *Ready*, so *Next* is enabled. By $\textit{WF_vars}(\textit{Next})$, *Next* must eventually fire. After it fires, either verdict became terminal (we are done) or attempted-count became $k+1$. Iterating gives termination in at most 9 transitions.

We formalize this as a leads-to chain on $P(k) := \textit{Terminated} \wedge \textit{AttemptedCount} = k$. Then:

- (a) *Init* satisfies $P(0)$ (vacuously: $0 = 0$).
- (b) For each k in $0 \dots 8$: $\textit{Spec} \ P(k) \ P(k+1)$, by one application of the *WF1* inference rule with witness action *Next*.
- (c) $P(9)$ implies *Terminated*: nine attempted steps and *verdict* = *UNKNOWN* is impossible — the only way for the ninth attempt to be a success is via *Check_JSON_DOC_CLASS_LEGAL_SUCCESS*, which sets *verdict* := *MUST_DO*; the only other option is a *Fail*(-), which sets *verdict* := *NOT_MUST_DO*.

The proof below states the leads-to chain. The individual links are flagged OMITTED — they are routine *WF1* applications and are the expected locus of *TLAPS* iteration; see properties.1.3.0.md, §“T3: Termination”, for the full *WF1* instances.

LEMMA *InitImpliesP0* $\triangleq \text{Init} \Rightarrow P(0)$

P(0) says *Terminated* $\vee$ *AttemptedCount* ≥ 0 . The right disjunct holds in every reachable state because *Cardinality* returns a natural and every natural is ≥ 0 . The proof reduces to invoking the standard finiteness lemmas from *TLAPS' FiniteSetTheorems* library; we keep the proof short and let *TLAPS/Toolbox* supply the cardinality bookkeeping. If iteration is needed, instantiate *FS_CardinalityType* on the set $\{s \in \text{Step} : \text{attempted}[s]\}$.

BY DEF *Init*, *P*, *Terminated*, *AttemptedCount*

LEMMA *P9ImpliesTerminated* $\triangleq \text{TypeOK} \wedge P(9) \Rightarrow \text{Terminated}$

$\langle 1 \rangle$ SUFFICES ASSUME *TypeOK*, *P(9)*, $\neg \text{Terminated}$

PROVE FALSE

OBVIOUS

P(9) and $\neg \text{Terminated}$ together force *AttemptedCount* $= 9$. Step has exactly 9 elements, so $\{s \in \text{Step} : \text{attempted}[s]\} = \text{Step}$, i.e. every step has been attempted. The 9th attempt was either the final success (which sets *verdict* = "MUST_DO") or some *Fail* (which sets *verdict* = "NOT_MUST_DO"). Either way *verdict* = "UNKNOWN", which means *Terminated* — contradicting the hypothesis $\neg \text{Terminated}$.

The cardinality bookkeeping below is routine. If *TLAPS* struggles on it, the auxiliary *FS_** lemmas from *FiniteSetTheorems* can be invoked explicitly; see *properties.1.3.0.md*, §“T3 supporting lemmas”.

$\langle 1 \rangle$ QED

BY DEF *TypeOK*, *Type*, *Step*, *P*, *AttemptedCount*, *Terminated*, *Verdict*

The leads-to links. Each is one application of the *WF1* rule with the witness action *Next*, the transient predicate “exactly *k* attempted”, and the goal “*k*+1 attempted or terminated”. The OMITTED tags are the intended iteration points in the TLA *Toolbox*.

LEMMA *Step0_1* $\triangleq \text{Spec} \Rightarrow (P(0) \leadsto P(1))$

PROOF OMITTED

LEMMA *Step1_2* $\triangleq \text{Spec} \Rightarrow (P(1) \leadsto P(2))$

PROOF OMITTED

LEMMA *Step2_3* $\triangleq \text{Spec} \Rightarrow (P(2) \leadsto P(3))$

PROOF OMITTED

LEMMA *Step3_4* $\triangleq \text{Spec} \Rightarrow (P(3) \leadsto P(4))$

PROOF OMITTED

LEMMA *Step4_5* $\triangleq \text{Spec} \Rightarrow (P(4) \leadsto P(5))$

PROOF OMITTED

LEMMA *Step5_6* $\triangleq \text{Spec} \Rightarrow (P(5) \leadsto P(6))$

PROOF OMITTED

LEMMA *Step6_7* $\triangleq \text{Spec} \Rightarrow (P(6) \leadsto P(7))$

PROOF OMITTED

LEMMA *Step7_8* $\triangleq \text{Spec} \Rightarrow (P(7) \leadsto P(8))$

PROOF OMITTED
 LEMMA *Step8_9* $\triangleq Spec \Rightarrow (P(8) \rightsquigarrow P(9))$
 PROOF OMITTED

THEOREM *Termination* $\triangleq Spec \Rightarrow \Diamond Terminated$
 $\langle 1 \rangle 1. Spec \Rightarrow (P(0) \rightsquigarrow P(9))$

Transitivity of $\rightsquigarrow$ chained nine times.

BY *Step0_1, Step1_2, Step2_3, Step3_4, Step4_5,*
Step5_6, Step6_7, Step7_8, Step8_9, PTL

$\langle 1 \rangle 2. Spec \Rightarrow P(0)$

BY *InitImpliesP0, PTL* DEF *Spec*

$\langle 1 \rangle 3. Spec \Rightarrow \Diamond P(9)$

BY $\langle 1 \rangle 1, \langle 1 \rangle 2, PTL$

$\langle 1 \rangle 4. Spec \Rightarrow \Box TypeOK$

BY *TypeCorrectness*

$\langle 1 \rangle 5. Spec \Rightarrow \Diamond Terminated$

BY $\langle 1 \rangle 3, \langle 1 \rangle 4, P9ImpliesTerminated, PTL$

$\langle 1 \rangle$ QED BY $\langle 1 \rangle 5$

END OF PROOFS

\ * Modification History
 \ * Last modified *Thu Apr 30 10:23:37 CEST 2026* by *gcordier*
 \ * Last modified *Thu Apr 30 2026* by Jean-Gabriel *Cordier* (with *Claude*)
 \ * Kaizen step 1.3.0 — added *TLAPS* proof bodies for *T1 (TypeCorrectness)*,
 \ * *T2 (VerdictStability)*, *T4 (XORCompleteness)*, *T5 (MustDoSoundness)*,
 \ * *T6 (NotMustDoSoundness)*, and a structured leads-to skeleton for
 \ * *T3 (Termination)* with nine OMITTED *WF1* links to be discharged in
 \ * the TLA *Toolbox*. Introduced the auxiliary state invariant
 \ * *Inv_ProvenAttempted* and the bridging lemma *ProvenAttemptedInvariant*.
 \ * Added *Naturals* to EXTENDS for *AttemptedCount/Cardinality* arithmetic.
 \ * Last modified *Wed Apr 29 2026* by Jean-Gabriel *Cordier* (with *Claude*)
 \ * Kaizen step 1.2.3 — proof-state semantics, full 9-step chain,
 \ * per-step *attempted record*, *MUST_DO/NOT_MUST_DO verdict*,
 \ * *XOR – completeness theorem, weak fairness*.
 \ * Created *Tue Aug 08 2023* by *gcordier*